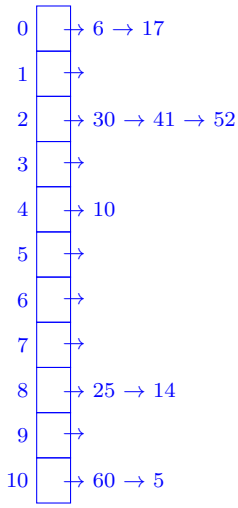
Informatics II, Spring 2026, Exercise 9

Publication of exercise: May 4, 2026
Publication of solution: May 11, 2026
Exercise classes: May 11 - 15, 2026

Task 1 (Easy)

Consider a hash table with 11 slots, and use chaining to resolve conflicts. For the hash function $h_1(k)$ below, draw the hash table after inserting the following values in the given order: 6, 30, 41, 25, 17, 10, 60, 14, 52, 5. *Note: Insert newer elements at the end of the linked list.*

(a) $h_1(k) = (k + 5) \mod 11$



Task 2 (Easy)

Now consider a Hash Table with open addressing. Insert the values {6, 30, 41, 25, 17, 10, 60, 15, 52, 5} using the provided hash functions $h_3(k, i)$ and $h_4(k, i)$.

(a) $h_3(k, i) = (k + i) \mod 11$

Slot	Value
0	52
1	5
2	
3	25
4	15
5	60
6	6
7	17
8	30
9	41
10	10

(b) $h_4(k,i) = (k \mod 11 + i * (k \mod 7)) \mod 11$

Slot	Value
0	52
1	
2	5
3	41
4	15
5	60
6	6
7	25
8	30
9	17
10	10

Task 3: Implement hash tables in C (Medium-Hard)

Implement a hash table with m slots, where m is a positive integer. All keys are strict positive integers. Use open addressing with linear probing with step size of 1 to resolve collisions:

$$h(k,i) = (3k + i) \mod m$$

The following hints will help you through the implementation.

(a) Define the number m that defines the size of the hash table. Set m to 7.

```
# define m 7
```

(b) The function `void init(int A[])` fills all slots of the hash table A with the value 0 (default value for an empty slot).

```
void init(int A[]) {
  int i;
  for (i = 0; i < m; i++) {
    A[i] = 0;
  }
}
```

(c) The hashing function `int h(int k, int i)` that receives the key k and the probe number i and returns the hashed key.

```
int h(int k, int i) {
  return (int)(3*k + i) % m;
}
```

(d) The function `void insert(int A[], int key)` inserts the key into the hash table A.

```
void insert(int A[], int key) {
  int counter = 0;
  int hkey;
  do {
    hkey = h(key, counter);
  } while(A[hkey] != 0 && counter++ < m);
  if (A[hkey] != 0) {
    printf("HT is full! Insertion not possible.\n");
    return;
  }

  A[hkey] = key;
}
```

(e) The function `int search(int A[], int key)` that returns -1 if the key was not found in the hash table A. Otherwise, it should return the index of the key in the hash table.

```
int search(int A[], int key) {
  int counter = 0;
  int hkey;
  do {
    hkey = h(key, counter);
  } while(A[hkey] != key && A[hkey] != 0 && counter++ < m);

  if (A[hkey] == key) return hkey;
  else return -1;
}
```

(f) The function `void printHash(int A[])` prints the table size and all slots of the hash table A accompanied with their index and the key.

```
void printHash(int A[]) {
  int i;
  printf("Table size: %d\n", m);
  for (i = 0; i < m; i++) {
    printf("i: %d\t key: %d\n", i, A[i]);
  }
}
```

For testing purposes, use a table size of 7, add the values 4, 9, 11, 16 & 5 and print your hash table. Search for the values 5 & 10 and print the results.

- *Search(HT,5)* should return 2
- *Search(HT,10)* should return -1

```
Output printHash:
Table size:  7
i:  0     key:  11
i:  1     key:  16
i:  2     key:  5
i:  3     key:  0
i:  4     key:  0
i:  5     key:  4
i:  6     key:  9
```

(g) The C code function `int HTDelete(int k)`, should delete key k from the hash table and also rearrange the other elements. The rearrangement leaves the table exactly as it would have been had key k never been inserted i.e., you cannot simply set the value to 0.

(a) What will the following hash table look like after deleting key 5. The hash function h given below was used for insertion.

Slot	0	1	2	3	4	5	6	7	8	9	10
Value	0	0	0	1	5	12	9	6	13	0	0

$h(k, i) = (3k + i) \mod 11$

Slot	0	1	2	3	4	5	6	7	8	9	10
Value	0	0	0	1	12	9	13	6	0	0	0

(b) Implement the function `int HTDelete(int k)`. Make sure to leverage the functions you have created in the previous steps. For testing purposes delete 9 & 11 after the insertions from above.

```
int HTDelete(int A[], int k){
        int i=0;
        int probe = h(k,i);
        int actualHashIdx= probe;
        while(i<m && A[probe] != 0 && A[probe]!=k){
                i++;
                probe = h(k,i);
        }
        // k not in A
  if(i>=m || A[probe]==0) return -1;

  // -> A[probe] == k
        A[probe] = 0;
        int j=(probe+1)%m;
        while(j!=actualHashIdx && A[j]!=0){
                if(h(A[j],0)!=j){
                        int tmpKey = A[j];
                        A[j] = 0;
            insert(A, tmpKey);
            printf("Reinserted key: %d\n", tmpKey);
                }
                j = (j+1)%m;
        }
        return probe;
}
```

Hash table after delete(HT,9):

```
Table size:  7
i:  0     key:  16
i:  1     key:  5
i:  2     key:  0
i:  3     key:  0
i:  4     key:  0
i:  5     key:  4
i:  6     key:  11
```

Hash table after delete(HT,11):

```
Table size:  7
i:  0     key:  0
i:  1     key:  5
i:  2     key:  0
i:  3     key:  0
i:  4     key:  0
i:  5     key:  4
i:  6     key:  16
```

Task 4 (Medium)

Describe how hash tables could be used to solve the following problems. Provide a small explanation and argumentation, followed by a pseudocode implementation. You may assume that basic functions to perform hash table operations have been implemented for you (insert, search, delete), similar to your implementations in Task 3.

(a) **Duplicates**
You are given two arrays A and B with respective lengths n and m containing strictly positive integers. Write an algorithm that prints to the console all elements that appear in both arrays ($A \cap B$). The algorithm should run in $O(n+m)$.

Hash tables work well for existence checks (exact match): Checking if a is in HT is the same as verifying if $search(a) \neq -1$. All hash table operations can be performed in $O(1)$ assuming a good hash function is used (thus no collisions appear).
To solve this problem, we can insert all items of A into an HT, and for each item of B, verify whether it was already inserted and thus must be in A.

Algo: `duplicates(A, B, n, m)`

```
1 HT[n]
2 init(HT)
3 for i = 1 to n do
4 |  insert(HT, A[i])
5 for j = 1 to m do
6 |  res = search(HT, B[j])
7 |  if res ≠ −1 then
8 |  |  print(B[j])
```

Complexity analysis:

- $init(), insert(), search()$ can be performed in $O(1)$
- First loop runs n times, $n \cdot 1 = n \Rightarrow O(n)$
- Second loop runs m times, $m \cdot 1 = m \Rightarrow O(m)$

$\Rightarrow$ $O(n) + O(m) = O(n+m)$ as both array sizes n, m are relevant to define the run time given the inputs A, B, n, m.

(b) **Target Sum** You are given an array A with n strictly positive, distinct integers and a target sum t. Write an algorithm to find if any two integers in A form the target sum t. The algorithm should run in $O(n)$ time and return 1 if it is possible to form t, and 0 otherwise.

As t is given, we can for each element a search for the required difference $b = t - a$. As above, one can insert all elements a and always first check whether $t - a$ is already in the hash table.
We check if $t - a$ is in the hash table before inserting a, because if $t - a = a$ and one would insert a first, the *search* of $t - a$ would match the just inserted a $\Rightarrow$ same element would be used twice to create target sum t

Algo: `target_sum`(A, n, t)

```
1 HT[n]
2 init(HT)
3 for i = 1 to n do
4 |  complement = t − A[i]
5 |  if search(HT, complement) ≠ −1 then
6 |  |  return 1
7 |  insert(HT, A[i])
8 return 0
```

Complexity analysis:

- $init(), search(), insert()$ run in $O(1)$
- The loop runs at most for n times
- n times constant work $\Rightarrow O(n)$